ALGORITHMES D'APPRENTISSAGE

On cache sous le terme « intelligence artificielle »un ensemble de techniques permettant à des machines d'accomplir des tâches et de résoudre des problèmes normalement réservés à des cerveaux humains, comme par exemple reconnaître des visages dans une image, jouer aux échecs, conduire une voiture, reconnaître une pathologie à partir d'un IRM, faire une dissertation...
Une des techniques en fort développement ces dernières années est **l'apprentissage automatique** (*machine learning*), dont l'essor est dû à l'augmentation considérable à la fois de la puissance de calcul, mais aussi du volume de données disponibles.
L'objet de ce chapitre est de découvrir deux modes d'apprentissage, le premier dit supervisé, et le deuxième dit non supervisé.

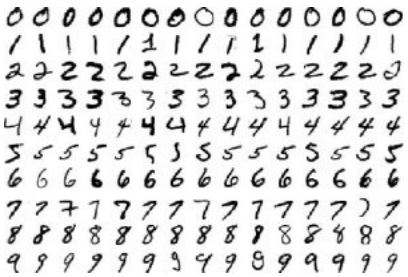
1 Apprentissage supervisé - Algorithme des k plus proches voisins

Pour aborder cette première technique d'apprentissage, nous allons nous intéresser à **l'algorithme des k plus proches voisins** (*k-nearest neighbors* en anglais, abrégé k-NN).
On dispose ici de données (qualifiées de données d'apprentissage) pour lesquelles une classe a été attribuée (par exemple, les données sont des photos, et les classes sont « photo de chien »et « photo de chat »). On cherche alors à construire une fonction capable de classifier de nouvelles données (déterminer si une nouvelle photo est une photo de chien ou de chat).

1.1 Un premier exemple

On souhaiterait qu'une machine puisse reconnaître le chiffre écrit dans l'image x suivante (pour reconnaître une plaque minéralogique, ou le montant écrit sur un chèque par exemple) :

On peut pour cela considérer une banque de 150 images, comme celle ci-dessous, dont on connait déjà les chiffres associés. On cherche alors les k images les plus proches (au sens d'une certaine distance) de l'image x dans la banque.



Par exemple, avec $k = 5$ dans la banque précédente on trouve quatre 3 et un 9 en plus proches voisins : on peut dire que x est un 3 ! On parle ici *d'apprentissage supervisé*, car on part d'une base de données où les classes d'appartenance sont connues.

1.2 Principe de l'algorithme k-NN

Etant donné un ensemble de données E déjà étiquetées et une nouvelle donnée q.

- Calculer la distance (euclidienne) entre la nouvelle donnée q et chaque donnée de l'ensemble de référence E.
- Trier les données de E selon leur distance à q, par ordre croissant, et garder les k premières (pour un k fixé à l'avance).
- Attribuer à q l'étiquette la plus fréquente parmi les k données les plus proches.

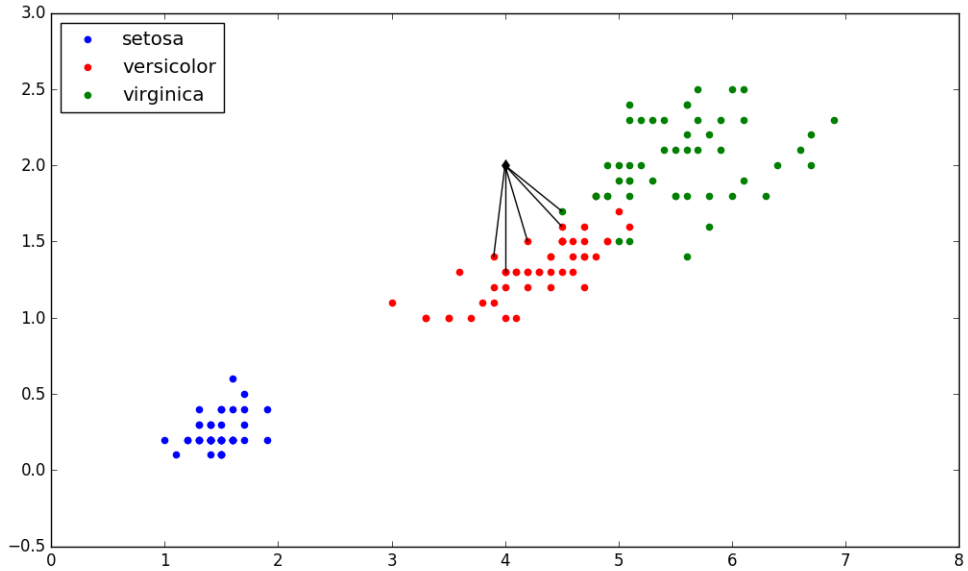


FIGURE 1 – La nouvelle donnée est classée « versicolor »car parmi ses 5 plus proches voisins 4 sont « versicolor »et 1 est « virginica ».

Exercice de révision : Comment se nomme le tri dont le code est donné ci-dessous ? Quel est sa complexité temporelle dans le pire des cas ? Est-il adapté à l'algorithme k-NN ? Adapter le code ci-dessous pour qu'il prenne comme paramètre une liste de couples et qu'il renvoie la liste triée selon la première coordonnée.

```
def tri(L):
    n=len(L)
    for i in range(1,n):
        j=i
        temp=L[i]
        while j>0 and temp<L[j-1]:
            L[j]=L[j-1]
            j=j-1
        L[j]=temp
    return(L)
```

Remarque : dans la suite, on utilisera la méthode `sort`, qui trie les couples selon l'ordre lexicographique.

1.3 Mise en œuvre sur un exemple

On a pris les mesures de **4 caractéristiques** de 100 fleurs d'Iris appartenant à **deux espèces** (Iris versicolor et Iris virginica), et on a enregistré les résultats dans le fichier `iris.txt`. Chaque ligne du fichier contient des données, séparées par une virgule.
On donne, ci-dessous, les trois premières lignes du fichier `iris.txt` :

```
sepal_length,sepal_width,petal_length,petal_width,species
5.1,3.5,1.4,0.2,setosa
4.9,3.0,1.4,0.2,setosa
```

On commence par construire la liste `E` de données étiquettées, puis on écrit une fonction permettant de déterminer l'espèce d'une fleur dont on a mesuré les caractéristiques :

```
MyFile=open('iris.txt','r')
MyFile.readline()#on passe la première ligne
```

```python
E=[]
for ligne in MyFile:
    ligne=ligne.strip('\n')
    L=ligne.split(',')
    E.append([[float(L[0]),float(L[1]),float(L[2]),float(L[3])],L[4]])
MyFile.close()

def dist(a,b):
    d=len(a)
    s=0
    for i in range(d):
       s+=(a[i]-b[i])**2
    return(sqrt(s))

def kNN(k,q,E):
    L=[(dist(q,x[0]),x[1]) for x in E]
    L.sort()
    r1,r2=0,0
    for i in range(k):
        if L[i][1]=='versicolor':
            r1+=1
        else:
            r2+=1
    if r1>r2:
        return('versicolor')
    else:
        return('virginica')

print(kNN(21,[6,3,0.75,2]))
```

Exercice : On suppose maintenant que les données d'apprentissage contiennent trois espèce de fleurs. Que faut-il modifié dans le script précédent ?

1.4 Matrice de confusion

Comment peut-on évaluer les résultats obtenus ? Et en particulier y a-t-il des meilleurs choix de k que d'autres ?

Dans la pratique, il est courant de réserver une partie des données d'apprentissage en vue de tester la qualité de l'algorithme. Revenons sur l'exemple précédent. On découpe aléatoirement la base initiale, qui contenait 50 fleurs de chaque classe en deux bases (une d'apprentissage et une de test) de 25 fleurs par classe. Et on lance l'algorithme pour chaque fleurs de test. Les résultats se répartissent alors en quatre catégories :

- L'algorithme renvoie `'versicolor'` et la vraie espèce est `'versicolor'` (VSVS fois sur les 50 tests)
- L'algorithme renvoie `'virginica'` et la vraie espèce est `'virginica'` (VGVG fois sur les 50 tests)
- L'algorithme renvoie `'virginica'` et la vraie espèce est `'versicolor'` (VSVG fois sur les 50 tests)
- L'algorithme renvoie `'versicolor'` et la vraie espèce est `'virginica'` (VGVS fois sur les 50 tests)

On range alors ces quatre valeurs dans une matrice 2×2, appelée **matrice de confusion** : $\begin{pmatrix} VSVS & VSVG \\ VGVS & VGVG \end{pmatrix}$

Plus la matrice de confusion est proche d'une **matrice diagonale**, plus *a priori* le choix de k est pertinent.

k	3	7	21
M	$\begin{pmatrix} 23 & 2 \\ 2 & 23 \end{pmatrix}$	$\begin{pmatrix} 23 & 2 \\ 1 & 24 \end{pmatrix}$	$\begin{pmatrix} 24 & 1 \\ 3 & 22 \end{pmatrix}$

FIGURE 2 – Matrices obtenues avec différentes valeurs de k pour l'exemple précédent.

Exercice : Reprendre l'exemple précédent et écrire les instructions permettant d'obtenir les matrices de confusion pour différentes valeurs de k.

Remarque : Dans le cas où il y a p classes, la matrice de confusion est de taille $p \times p$.

2 Apprentissage non supervisé - Algorithme des k-moyennes

Pour cette deuxième technique d'apprentissage, nous allons nous intéresser à **l'algorithme des k-moyennes** (*k-means clustering* en anglais). Il s'agit ici de partitionner automatiquement des données, en k classes.
Par exemple, en regardant la figure ci-dessous nous voyons trois classes de points. Il faudrait faire en sorte qu'il en soit de même pour la machine. Nous allons l'aider un peu en lui imposant le nombre k de classes.

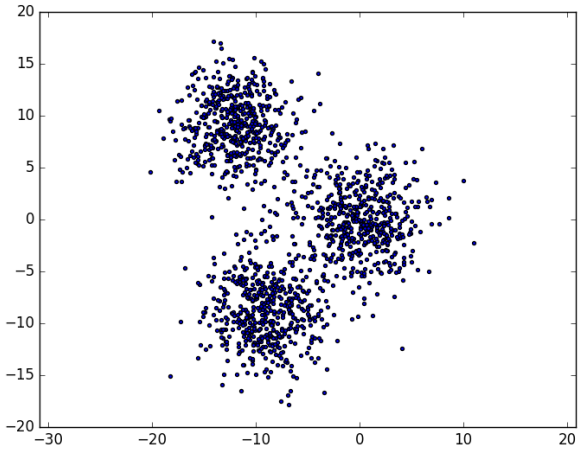


FIGURE 3 – Les données semblent se regrouper en trois classes

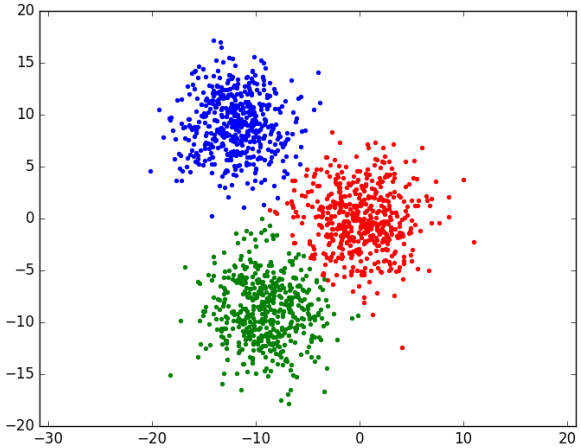
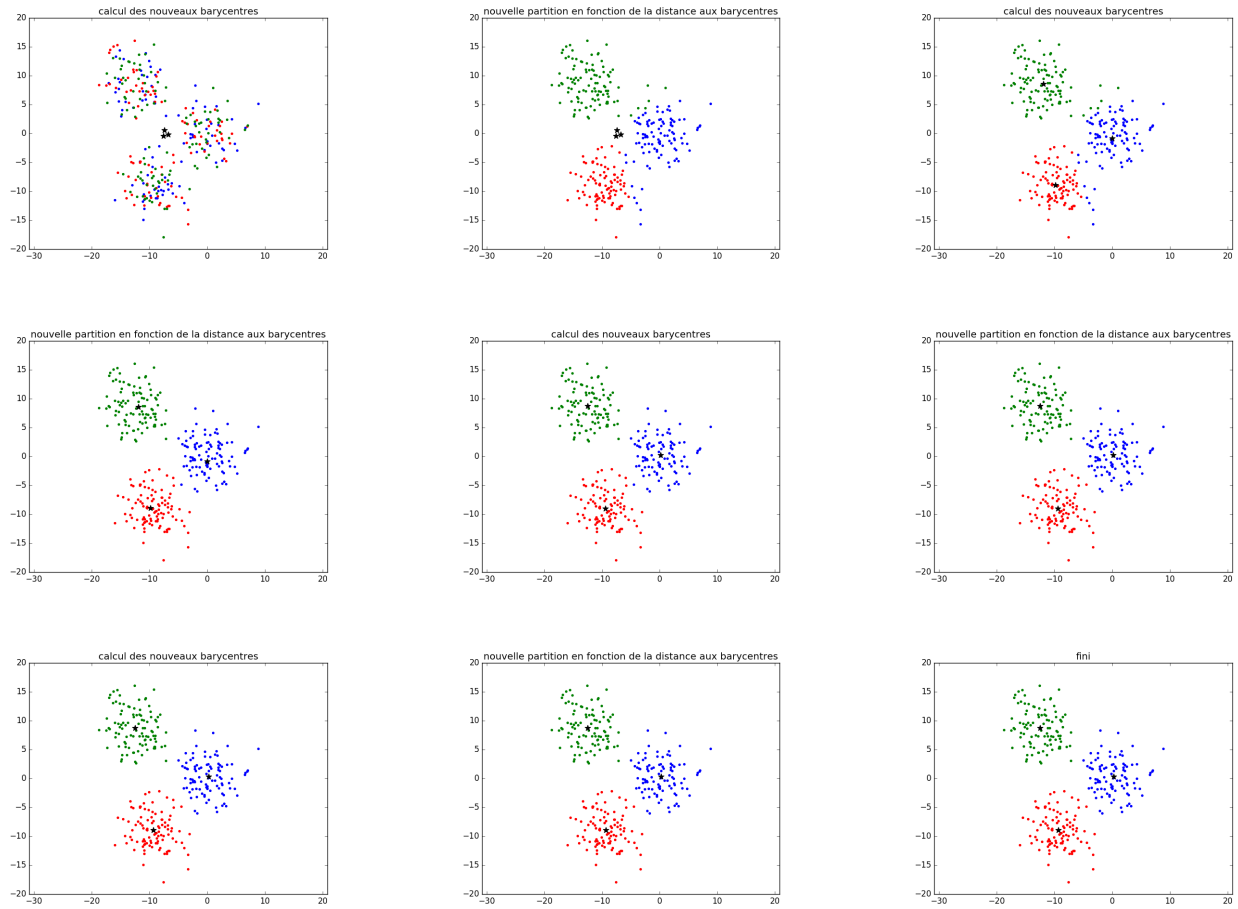


FIGURE 4 – Le résultat de l'algorithme des k-moyennes pour $k = 3$

2.1 Principe

Disons pour simplifier que les classes sont des couleurs. L'algorithme des k-moyennes va attribuer une couleur à chaque point selon le principe suivant :

1. Attribuer aléatoirement une couleur à chaque point
2. Déterminer le barycentre de chaque groupe de couleur
3. Attribuer à chaque point la couleur du barycentre le plus proche
4. Recommencer les étapes 2 à 4 jusqu'à ce qu'aucun point ne change plus de couleur.



Remarque :

- On peut montrer que l'algorithme converge, c-à-d qu'après un nombre fini d'étapes les barycentres de changent plus.
- L'algorithme n'interprète rien et ne fait qu'essayer de minimiser la somme des moments d'inertie (c-à-d $\sum_{i=1}^{k} \sum_{x \in C_i} ||x - \bar{x}_i||^2$).
- L'algorithme ne donne pas forcément la classification optimale, et le résultat peut changer suivant le choix initial. Mais il converge toujours, vers ce qui est parfois appelé des minimums locaux (au sens où les étapes ne modifient plus la répartition).

FIGURE 5 – Deux partitions sur lesquelles l'algorithme n'évolue plus. Une optimale, l'autre non.

2.2 Mise en œuvre

```
from math import *
import random as rd
def dist(a,b):
    """distance euclidienne dans le plan"""
    xa,ya=a
    xb,yb=b
    return(sqrt((xa-xb)**2+(ya-yb)**2))

def barycentre(X,xi):
    """renvoie le barycentre des X[k] pour k dans xi"""
    s=[0,0]
    for k in xi:
        s[0]+=X[k][0]
        s[1]+=X[k][1]
    s[0]=s[0]/len(xi)
    s[1]=s[1]/len(xi)
    return(s)

def initialisation(n,k):
    """renvoie une partition de [0,n-1] en k parties non vides"""
    L=list(range(n))
    rd.shuffle(L)
    X=[[] for i in range(k)]
    for i in range(k):
        X[i].append(L[i])
    for i in range(k,n):
        X[rd.randrange(0,k)].append(L[i])
    return(X)

def plusproche(p,M):

    i,d=0,dist(p,M[0])
    for j in range(1,len(M)):
        if dist(p,M[j])<d:
            i,d=j,dist(p,M[j])
    return(i)

def kMean(X,k):

    n=len(X)
    part=initialisation(n,k)
    while True:
        M=[barycentre(X,xi) for xi in part]
        part2=[[] for i in range(k)]
        for i in range(n):
            part2[plusproche(X[i],M)].append(i)
        if part==part2:
            return part
        else:
            part=part2
```

Remarque :

- L'utilisation de `numpy` permettrait de simplifier quelques fonctions.
- Il serait intéressant de tester cet algorithme sur le tableau de chiffres du paragraphe 1.1, avec $k = 10$.
- On peut adapter cet algorithme à la compression d'image, l'objectif étant de réduire le nombre de couleur utilisées à k. Après application de l'algorithme, on remplace chacune des couleurs de l'image initiale par le barycentre de sa classe.